

Functional Documentation & Unit Testing

Testing in Android with Kotlin

Understanding Functional Testing: Methods, Timing & Tools



Why Testing Matters

- Catch bugs early, before users do
- Make code easier to maintain and refactor
- Provide a safety net for new features
- Increase developer confidence
- Enable Continuous Integration (CI) and automation



Real-life example:

Imagine your login button stops working after a UI update. A test would catch that before release!

Types of Android Tests

Test Type	Speed	Scope	Runs On
Unit Test	⚡ Fast	Logic only	JVM (local machine)
Instrumented Test	🐢 Medium	Android context	Emulator or device
UI Test (Espresso)	🐢 🐢 Slower	Real UI interaction	Emulator or device

Structure

test/ – Unit tests

androidTest/ – Instrumented and UI tests

Setup in build.gradle

```
// app/build.gradle
dependencies {
    testImplementation "junit:junit:4.13.2"
    androidTestImplementation "androidx.test.ext:junit:1.1.5"
    androidTestImplementation "androidx.test.espresso:espresso-core:3.5.1"
}
```

🔧 Add plugins if needed:

```
plugins {
    id 'com.android.application'
    id 'kotlin-android'
}
```


Unit Test Example

Calculator.kt

```
class Calculator {  
    fun add(a: Int, b: Int): Int = a + b  
}
```

CalculatorTest.kt

```
class CalculatorTest {  
    @Test  
    fun testAddition() {  
        val calc = Calculator()  
        assertEquals(5, calc.add(2, 3))  
    }  
}
```


Key Points

- Fast and local
- Does not require Android SDK

Real Unit Test – ViewModel Logic

LoginViewModel.kt

```
class LoginViewModel {  
    fun isValidEmail(email: String): Boolean {  
        return email.contains("@")  
    }  
}
```

LoginViewModelTest.kt

```
class LoginViewModelTest {  
    private val viewModel = LoginViewModel()  
  
    @Test  
    fun validEmail_returnsTrue() {  
        assertTrue(viewModel.isValidEmail("test@mail.com"))  
    }  
  
    @Test  
    fun invalidEmail_returnsFalse() {  
        assertFalse(viewModel.isValidEmail("invalid.com"))  
    }  
}
```

📌 **Real Use Case:** Validating form inputs before network calls.

Instrumented Test

Used for logic that requires **Android context** (e.g. resources, SharedPreferences)

```
@RunWith(AndroidJUnit4::class)
class ContextTest {
    @Test
    fun useAppContext() {
        val context = ApplicationProvider.getApplicationContext<Context>()
        assertEquals("com.example.myapp", context.packageName)
    }
}
```

 **Use Case:** Testing resource access, database, or services tied to Android lifecycle.

Espresso UI Test – Basics

Tests actual UI like a user would.

```
@RunWith(AndroidJUnit4::class)
class LoginUITest {
    @Test
    fun loginButton_isDisplayed() {
        ActivityScenario.launch(LoginActivity::class.java)
        onView(withId(R.id.login_button)).check(matches(isDisplayed()))
    }
}
```

Checks:

- Views visible or clickable
- Text input working
- Buttons reacting properly

Real UI Test Scenario – Login Flow

@Test

```
fun loginFlow_showsWelcomeScreen() {  
    ActivityScenario.launch(LoginActivity::class.java)  
  
    onView(withId(R.id.email_field)).perform(typeText("test@email.com"))  
    onView(withId(R.id.password_field)).perform(typeText("123456"))  
    onView(withId(R.id.login_button)).perform(click())  
  
    onView(withText("Welcome")).check(matches(isDisplayed()))  
}
```



Simulates full login interaction

Best Practices

Name tests clearly: givenX_whenY_thenZ

Write small, focused tests

Keep unit tests Android-free

Use mocking for dependencies (e.g., Mockito)

Run tests frequently

Run & Automate Tests

 Run locally:

bash

```
./gradlew test
```

```
./gradlew connectedAndroidTest
```

 In CI/CD (e.g., GitHub Actions):

yaml

```
- name: Run Unit Tests  
  run: ./gradlew test
```


Tools You Can Use

Tool	Use
JUnit	Unit testing framework
Mockito	Mock dependencies
Espresso	UI testing
Truth	Fluent assertions
Robolectric	JVM-based Android test runner

Mocking Example (with Mockito)

```
interface UserRepo {  
    fun getUsername(): String  
}  
  
class UserService(private val repo: UserRepo) {  
    fun greeting(): String = "Hello, ${repo.getUsername()}"  
}  
  
@Test  
fun testGreeting() {  
    val repo = mock(UserRepo::class.java)  
    `when`(repo.getUsername()).thenReturn("Ryan")  
  
    val service = UserService(repo)  
    assertEquals("Hello, Ryan", service.greeting())  
}
```

Wrap-Up & What to Practice

✓ You've learned:

- Unit tests (ViewModel, logic)
- Instrumented tests (Android context)
- UI tests (Espresso)
- Mocking and best practices

Practice Ideas:

- Test your login flow
- Validate user input
- Mock a repository
- Create CI pipeline with test step

Technical Documentation

Why Documentation Matters

Improves team communication and collaboration

Makes onboarding new developers faster

Helps future-you understand current-you

Enables smooth handoffs between devs, designers, QA, and managers

Prevents confusion, rewrites, and bugs

📌 Real-life example:

Your teammate leaves, and suddenly no one knows how the login state is cached. A single paragraph in the README could've saved days of guesswork.

Types of Documentation in Mobile Projects

Project Overview – what's this app, what problem does it solve?

Architecture Docs – MVVM, Clean Architecture, BLoC, etc.

API Documentation – endpoints used, headers, sample payloads

README.md – central doc for quickstart, setup, and structure

Build & Deployment Guides – how to build, sign, and release

Testing Strategy – what to test and how

Changelog/Release Notes – version history and what's new

Documentation in Android (Kotlin) Projects

✓ KDoc Comments: Use multi-line comments for Kotlin classes/functions:

```
/*  
 * Validates user input during login.  
 * @param input The email or phone number.  
 */  
fun validate(input: String): Boolean { ... }
```

 Tools:

Dokka for HTML docs

Use README.md and link to architecture diagrams (e.g., MVVM layers)

Include setup for Gradle, flavors, build variants

 Pro Tip:

Document Retrofit interfaces, data sources, and ViewModel responsibilities.

Example – README Checklist

Here's what a solid README.md should include:

- ✓ App Name and Short Description
 - ✓ Screenshots or screen recording
 - ✓ Setup Instructions (e.g., how to clone, run, build)
 - ✓ Folder Structure Overview
 - ✓ Tech Stack Used
(e.g., Kotlin + Hilt or Flutter + Riverpod)
 - ✓ API Keys or Environment Setup (use .env.example)
 - ✓ Contribution Guidelines
 - ✓ License Info
- 📌 Pro Tip:
Use badges (build passing ☒, coverage %) for credibility.

API Documentation Best Practices

For mobile apps that consume APIs, document the following:

-  Endpoint names and HTTP methods
-  Required headers, auth tokens
-  Sample request/response payloads
-  Error messages and retry logic
-  Versioning (e.g., /v1/, /v2/)
-  Link directly from service class or API client

 Tools:

Swagger / OpenAPI

Postman with shared collections

Redoc / Stoplight Studio

Tools to Help You Write Great Docs

Tool	Use Case
Dokka	Kotlin/Java code documentation
dartdoc	Flutter code documentation
Markdown (.md)	Universal for READMEs & wikis
Mermaid / PlantUML	Diagrams in Markdown
GitBook	Clean, shareable internal docs
Notion / Confluence	For team-wide documentation

Best Practices for Writing Docs

- 🧠 Write like your future self will read it
- 🌸 Keep it clean, short, and to the point
- 🔄 Update docs as code changes
- 🎯 Focus on why + how, not just what
- 📁 Use consistent folder structure (docs/, /designs, etc.)
- 🔗 Use links instead of duplicating content
- 📌 Pro Tip:
Use templates for each project — save time and ensure consistency.

Common Mistakes to Avoid

- ✗ No README at all
- ✗ Outdated documentation (last touched in 2021)
- ✗ Over-documenting trivial details (e.g., “This class has a variable”)
- ✗ Ignoring architecture and decisions
- ✗ No clear setup instructions

● Real Case:

Developers waste hours setting up Firebase manually when a 3-line setup section would’ve saved the day.

Wrap-Up & Action Steps

Start documenting today by doing the following:

-  Create or update your README.md
-  Add doc comments to classes, services, and utils
-  Write a brief explanation of your architecture
-  Document how to build, test, and release the app
-  Share testing strategy and API endpoints
-  Create a docs/ folder if your project lacks one
-  Documentation is code. Treat it like part of your product.